

SLE-3: Architectural Design (Full C4 Model)

Course: 02AML204 – Introduction to Artificial Intelligence

PRN: 25UAM080 Name: Ratan Ranjeet Sankpal

Division: B Date: _____

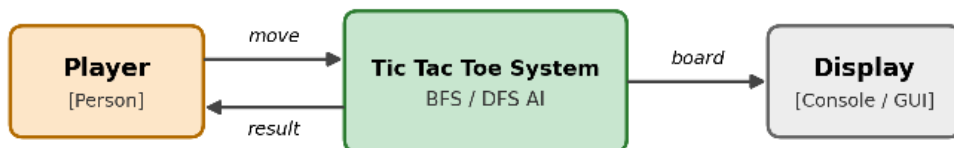
GitHub / Portfolio Link (optional): _____

1. System Title & Short Description

Tic Tac Toe AI System (BFS and DFS Search). This system lets a human player play Tic Tac Toe against an AI agent on a 3x3 board. The agent chooses its move using state-space search: Breadth-First Search (BFS) explores the game tree level by level and finds the quickest winning line, while Depth-First Search (DFS) follows one line as deep as possible and then backtracks. It solves the problem of making the computer find a winning move without any heuristic. This is the same system I coded in SLE-1 and profiled in SLE-2.

2. Context Diagram (Level 1)

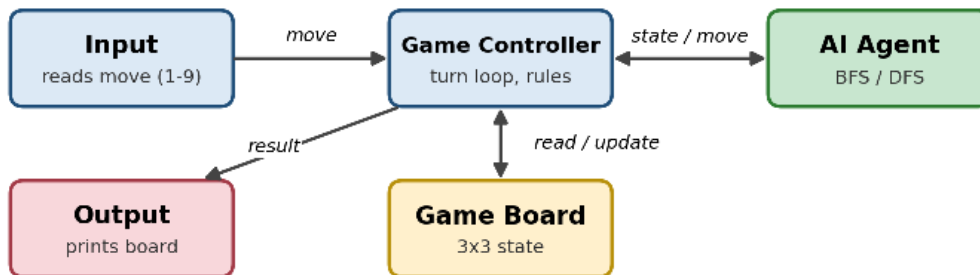
Level 1 - Context: Tic Tac Toe AI System



The Player enters a move (a cell number) and receives the updated board and the game result. The Tic Tac Toe System is the only software system in scope. It renders the board on an external console or GUI display. No other external services are needed, so the context stays simple.

3. Container Diagram (Level 2)

Level 2 - Containers: Tic Tac Toe AI System



Input Module: reads and validates the player's move (cell 1-9).

Game Controller: runs the turn loop, applies the rules, and checks for win or draw.

AI Agent: chooses the computer's move by searching game states with BFS (queue) or DFS (stack).

Game Board: stores the 3x3 state and supports legal moves, make move and undo move.

Output Module: prints the board, the moves played and the final result.

4. Component Diagram (Level 3)

Level 3 - Components: inside the AI Agent



I chose the AI Agent because it holds the core decision logic. The Successor Generator lists the

empty cells and builds the next boards. The Frontier holds the boards waiting to be checked: it is a queue for BFS (oldest first, level by level) and a stack for DFS (newest first, going deep). The Goal Test checks whether a board is an AI win. When a winning path is found, the Move Selector returns the first move of that path.

5. Code Level Overview (Level 4)

Class / Function	Responsibility
class Board	Stores the 3x3 grid; make_move, undo_move, get_empty_cells, winner, is_full
def bfs(board, player)	Breadth-First Search with a queue; expands level by level and returns the shortest winning move sequence
def dfs(board, player)	Depth-First Search with a stack or recursion; goes deep, backtracks with undo_move, returns the first winning line found
def is_goal(board)	Goal test: true for an AI win; a player win or a draw is a dead end
def best_move(board, method)	Runs BFS or DFS from the current board and returns the first move of the path found
def play_game()	Game loop: take player move, call AI, check result, print board
def print_board(board)	Displays the current board in the console

6. Design Decisions

- **Separated board from AI:** the AI Agent only reads and updates the Game Board through make/undo moves, so the rules can change without touching the search code.
- **Goal test as its own module:** the win / loss / draw check is in one place, so BFS and DFS both reuse it and it is easy to change.
- **BFS and DFS kept inside the AI Agent:** both share the same successor generator and goal test, so only the frontier (queue or stack) differs, which makes comparing them (as in SLE-2) fair.
- **Kept to 5 containers:** small enough to explain easily while still showing every major responsibility.

7. AI Contribution Note

AI tools used (if any): Claude (Anthropic).

What AI helped with: drafting the diagram layouts, the container/component descriptions and the wording of this report.

What I did myself: By taking the Code of Tic Tac Toe . I checked the

8. Conclusion

Time and Space complexity using python libraries

Using the C4 model, I learned to look at one system at four zoom levels: who uses it, what the main parts are, how one part works inside, and which classes and functions implement it. Splitting the Tic Tac Toe game into clear containers made it easier to see where rules, AI search and output belong. This structure makes the system simpler to test, compare (as in SLE-2) and extend to bigger games such as Connect Four.